小鸿天气应用 - 需求说明书与功能分析

一、需求说明书

1.1 应用背景与目标

应用名称： 小鸿天气（HarmonyOS Weather App）

应用版本： 1.0.0

技术平台： 鸿蒙操作系统（HarmonyOS）

开发工具： DevEco Studio 3.1.0.501

应用背景：
随着智能设备的普及，用户对天气信息的需求日益增长。小鸿天气应用基于鸿蒙操作系统开发，旨在为用户提供准确、全面、易用的天气信息查询服务，满足用户对天气预报、空气质量、生活指数等多维度信息的需求。

应用目标：

- 提供实时、准确的多城市天气信息
- 支持未来24小时逐小时天气预报
- 提供7天天气趋势预测
- 展示空气质量指数和生活建议
- 实现多城市管理和快速切换
- 提供桌面天气卡片（Widget）功能
- 打造简洁美观的用户界面

1.2 功能需求

1.2.1 核心功能

F1. 天气信息展示

- F1.1 实时天气信息：当前温度、天气状况、温度范围、空气质量
- F1.2 24小时天气预报：逐小时温度和天气变化趋势
- F1.3 7天天气预报：未来一周的天气、温度、风向风力
- F1.4 温度曲线图：可视化展示温度变化趋势
- F1.5 详细气象数据：风向风力、相对湿度、气压、体感温度、紫外线指数

F2. 空气质量监测

- F2.1 空气质量指数（AQI）显示
- F2.2 主要污染物监测：PM2.5、PM10、NO2、SO2、CO、O3
- F2.3 空气质量等级评定

F3. 生活指数建议

- F3.1 晨练指数
- F3.2 洗车指数
- F3.3 紫外线指数
- F3.4 感冒指数
- F3.5 晾晒指数
- F3.6 穿衣指数

F4. 多城市管理

- F4.1 城市搜索功能
- F4.2 热门城市快速选择
- F4.3 已保存城市列表管理（最多10个）
- F4.4 当前城市快速切换
- F4.5 城市删除功能

F5. 数据缓存与存储

- F5.1 天气数据本地缓存（30分钟有效期）
- F5.2 城市列表持久化存储
- F5.3 当前城市状态保存

1.2.2 新增功能

F6. 桌面小部件（Widget/卡片服务）

- F6.1 支持多种尺寸：2×2、2×4、4×4
- F6.2 卡片数据自动更新（每小时更新）
- F6.3 点击卡片跳转到主应用
- F6.4 卡片显示简要天气信息

F7. 启动加载页面

- F7.1 应用启动时显示加载进度
- F7.2 进度条动画效果
- F7.3 自动跳转到天气主页

F8. 用户交互优化

- F8.1 双击返回键退出应用
- F8.2 返回操作提示
- F8.3 城市切换即时反馈

- F8.4 数据加载状态提示

1.3 非功能需求

1.3.1 性能需求

NFR1. 响应时间

- 应用启动时间：≤3秒
- 页面切换响应：≤500ms
- 数据加载时间：≤2秒
- 城市切换响应：≤1秒

NFR2. 资源占用

- 内存占用：≤100MB
- 安装包大小：≤20MB
- CPU占用率：正常使用时≤10%

NFR3. 数据缓存

- 天气数据缓存时长：30分钟
- 离线可用性：支持缓存数据离线查看

1.3.2 安全性需求

NFR4. 数据安全

- 本地数据加密存储
- 网络数据传输安全（预留API接口）
- 用户隐私保护

NFR5. 稳定性

- 崩溃率：≤0.1%
- 异常处理机制完善
- 数据解析失败降级方案

1.3.3 可用性需求

NFR6. 用户体验

- 界面简洁美观，符合鸿蒙设计规范
- 操作流程简单，3步内完成主要功能
- 支持中英文国际化
- 响应式布局，适配不同屏幕尺寸

NFR7. 兼容性

- 支持鸿蒙手机和平板设备
- 设计宽度：720px
- 支持横竖屏切换

1.3.4 可维护性需求

NFR8. 代码质量

- 模块化设计，组件复用率≥80%
- 代码注释覆盖率≥30%
- 遵循JavaScript ES6+标准

NFR9. 可扩展性

- 支持天气数据源切换
- 支持新增城市数据
- 支持功能模块插拔

1.4 用户角色与使用场景

1.4.1 用户角色

R1. 普通用户

- 查看所在城市的天气信息
- 获取生活建议指数
- 查询未来天气趋势

R2. 多城市用户

- 管理多个关注城市
- 快速切换不同城市
- 对比不同城市天气

R3. 移动办公用户

- 通过桌面卡片快速查看天气
- 根据天气调整出行计划
- 获取空气质量信息

1.4.2 使用场景

S1. 早晨查看天气 用户起床后打开应用，查看当天天气和穿衣建议，决定着装方案。

S2. 出行前查询 用户计划外出前，查看未来几小时的天气变化，携带雨具或防晒用品。

S3. 旅游规划 用户计划去其他城市旅游，添加目的地城市，查看未来7天天气，安排行程。

S4. 空气质量关注 用户关注空气质量，决定是否进行户外运动或开窗通风。

S5. 快速浏览 用户在桌面通过卡片快速查看天气摘要，无需打开应用。

S6. 城市切换 出差用户到达新城市，切换城市查看当地天气，获取生活建议。

二、功能分析

2.1 功能模块划分

应用采用分层架构，主要分为以下功能模块：

2.1.1 表现层（UI Layer）

M1. 启动模块（Loading Module）

- 组件：`pages/loading/loading`
- 功能：显示应用启动加载动画和进度条

M2. 天气展示模块（Weather Display Module）

- 组件：`pages/weather/weather`
- 子组件：
  - `title`：城市名称和导航
  - `briefWeatherInfo`：天气概览（温度、天气状况、空气质量）
  - `timeWeather`：24小时天气列表
  - `temperatureChart`：温度曲线图
  - `weeklyWeather`：7天天气预报
  - `WindPointer`：风向风力显示
  - `rainCounter`：降水量和云量统计
  - `senseList`：体感温度、湿度、气压等信息
  - `qualityWeather`：空气质量详情
  - `livingIndex`：生活指数建议
  - `end`：页面底部信息

M3. 城市选择模块（City Selector Module）

- 组件：`pages/citySelector/citySelector`
- 功能：
  - 城市搜索
  - 热门城市展示
  - 已保存城市管理
  - 当前城市显示

M4. 卡片服务模块（Widget Module）

- 组件：`widget/pages/index/index`
- 功能：桌面天气卡片展示

2.1.2 业务逻辑层（Business Layer）

M5. 天气数据管理模块（Weather Data Manager）

- 文件：`utils/cityWeatherManager.js`
- 功能：
  - 天气数据获取和缓存
  - 模拟数据生成
  - 多城市天气管理
  - 数据格式化和转换

M6. 城市存储管理模块（City Storage Manager）

- 文件：`utils/cityStorageManager.js`
- 功能：
  - 当前城市管理
  - 已保存城市列表维护
  - 城市搜索
  - 热门城市数据提供

M7. 工具模块（Utility Module）

- 文件：`utils/imgSelect.js`
- 功能：天气图标映射和选择

2.1.3 数据层（Data Layer）

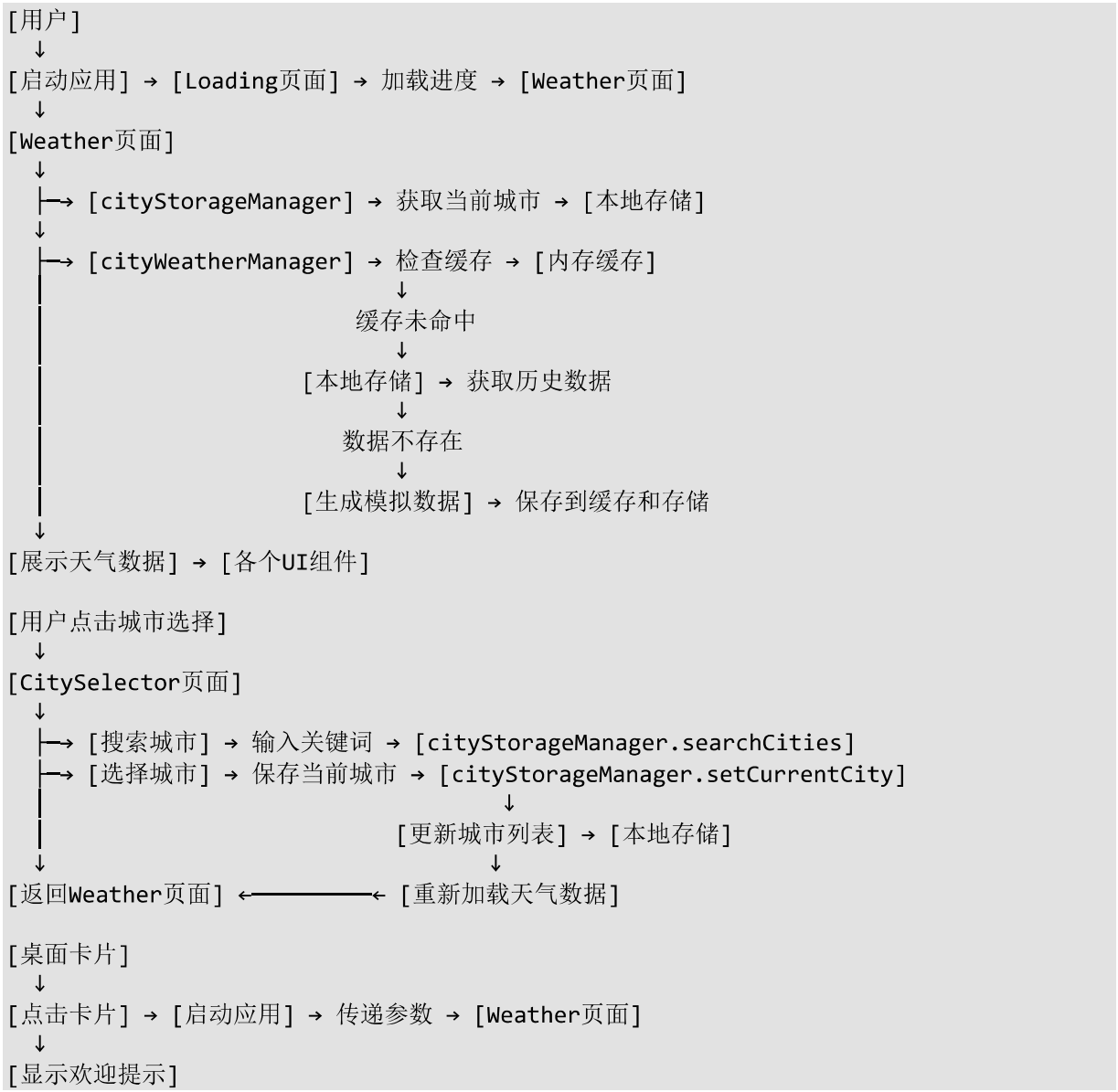
M8. 本地存储模块（Local Storage）

- 技术：`@system.storage`
- 功能：
  - 天气数据持久化
  - 城市列表存储
  - 用户偏好设置

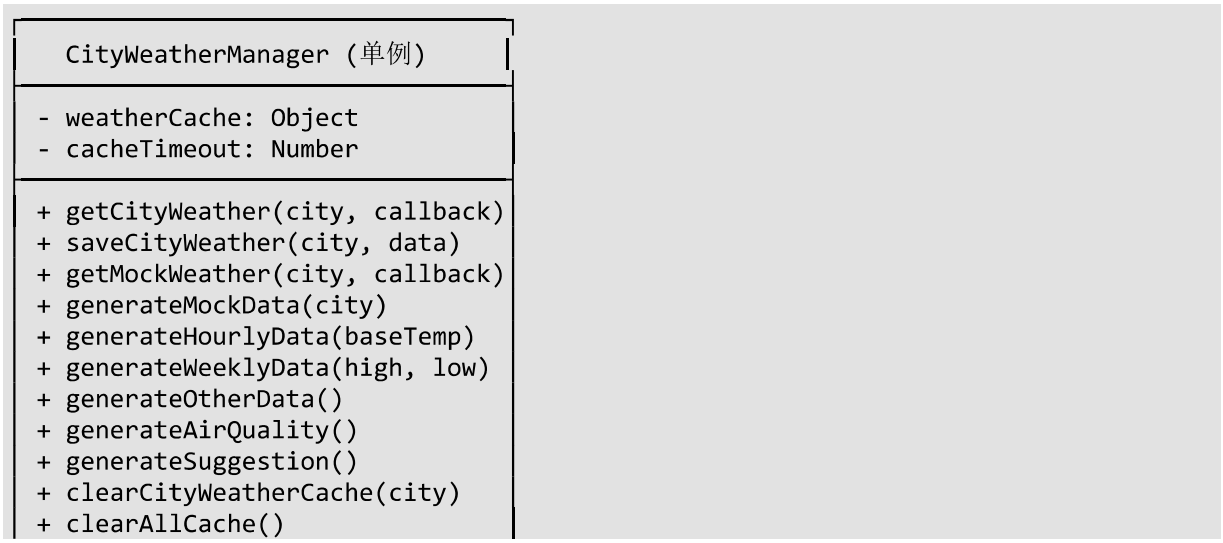
M9. 国际化资源模块（i18n Resources）

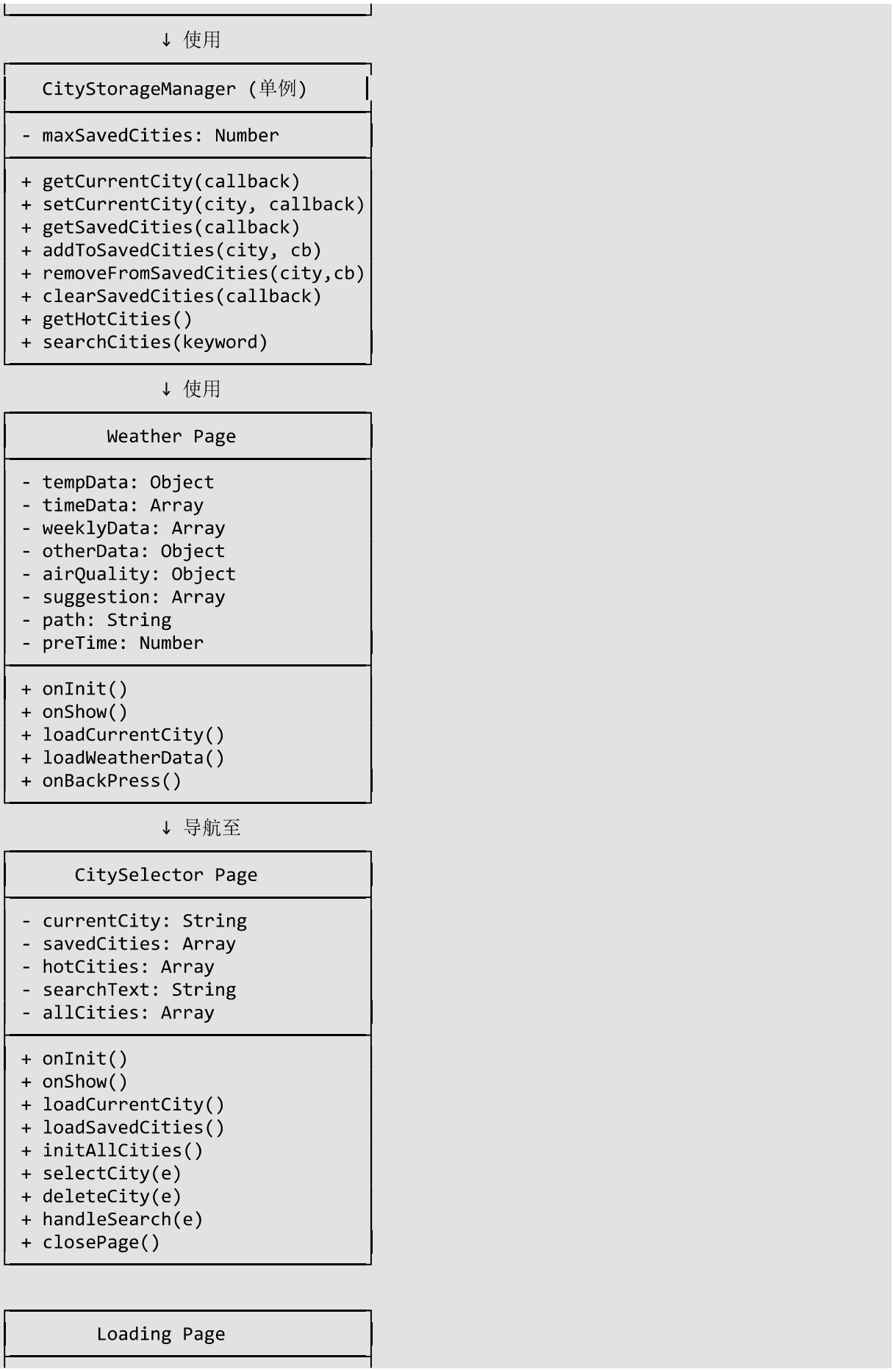
- 文件：`i18n/zh-CN.json`, `i18n/en-US.json`
- 功能：多语言支持

2.2 数据流图


[用户]
[启动应用] → [Loading页面] → 加载进度 → [Weather页面]
[Weather页面]
├→ [cityStorageManager] → 获取当前城市 → [本地存储]
├→ [cityWeatherManager] → 检查缓存 → [内存缓存]
缓存未命中
[本地存储] → 获取历史数据
数据不存在
[生成模拟数据] → 保存到缓存和存储
[展示天气数据] → [各个UI组件]
[用户点击城市选择]
[CitySelector页面]
├→ [搜索城市] → 输入关键词 → [cityStorageManager.searchCities]
├→ [选择城市] → 保存当前城市 → [cityStorageManager.setCurrentCity]
[更新城市列表] → [本地存储]
[返回Weather页面] ←——— [重新加载天气数据]
[桌面卡片]
[点击卡片] → [启动应用] → 传递参数 → [Weather页面]
[显示欢迎提示]

2.3 类图


CityWeatherManager (单例)
- weatherCache: Object
- cacheTimeout: Number
+ getCityWeather(city, callback)
+ saveCityWeather(city, data)
+ getMockWeather(city, callback)
+ generateMockData(city)
+ generateHourlyData(baseTemp)
+ generateWeeklyData(high, low)
+ generateOtherData()
+ generateAirQuality()
+ generateSuggestion()
+ clearCityWeatherCache(city)
+ clearAllCache()


↓ 使用
CityStorageManager (单例)
- maxSavedCities: Number
+ getCurrentCity(callback)
+ setCurrentCity(city, callback)
+ getSavedCities(callback)
+ addToSavedCities(city, cb)
+ removeFromSavedCities(city,cb)
+ clearSavedCities(callback)
+ getHotCities()
+ searchCities(keyword)
↓ 使用
Weather Page
- tempData: Object
- timeData: Array
- weeklyData: Array
- otherData: Object
- airQuality: Object
- suggestion: Array
- path: String
- preTime: Number
+ onInit()
+ onShow()
+ loadCurrentCity()
+ loadWeatherData()
+ onBackPress()
↓ 导航至
CitySelector Page
- currentCity: String
- savedCities: Array
- hotCities: Array
- searchText: String
- allCities: Array
+ onInit()
+ onShow()
+ loadCurrentCity()
+ loadSavedCities()
+ initAllCities()
+ selectCity(e)
+ deleteCity(e)
+ handleSearch(e)
+ closePage()
Loading Page

```
| - timeout: Number                  |
|------------------------------------|
| + onInit()                         |
| + updateProgress()                 |
| + navigateToPage()                 |
```

2.4 状态图

2.4.1 应用生命周期状态图

```
[应用启动]
    ↓
[Loading状态]
    │ 进度: 0% → 100%
    │ 持续时间: ~3秒
    ↓
[自动跳转]
    ↓
[Weather主页 - 活动状态]
    │
    ├→ [后台状态] ← 用户切换应用
    │      ↓
    │   [恢复] → [Weather主页 - 活动状态]
    │
    ├→ [城市选择状态] ← 用户点击城市选择
    │      │
    │      ├─ 搜索城市
    │      ├─ 选择城市
    │      ├─ 删除城市
    │      ↓
    │   [返回] → [Weather主页 - 活动状态]
    │
    └→ [退出应用] ← 双击返回键
```

2.4.2 天气数据加载状态图

```
[请求天气数据]
    ↓
[检查内存缓存]
    │
    ├─ 缓存有效 → [返回缓存数据] → [渲染页面]
    │
    └─ 缓存无效/不存在
        ↓
    [检查本地存储]
        │
        ├─ 存储有效 → [加载存储数据] → [更新内存缓存] → [渲染页面]
        │
        └─ 存储无效/不存在
            ↓
        [生成模拟数据]
            ↓
        [保存到存储和缓存]
            ↓
        [渲染页面]
```

2.4.3 城市切换状态图

```
[当前城市: A]
    ↓
[用户打开城市选择页]
    ↓
[显示城市列表]
    │
    ├─ [搜索] → 输入关键词 → [过滤城市列表] → [显示搜索结果]
    │
    └─ [选择城市B]
        ↓
    [保存城市B为当前城市]
        ↓
    [添加城市B到已保存列表]
        ↓
    [显示Toast提示: "已切换到城市B"]
        ↓
    [返回天气页面]
        ↓
    [清除旧天气数据]
        ↓
    [加载城市B天气数据]
        ↓
    [渲染新天气数据]
        ↓
    [当前城市: B]
```

2.5 技术选型说明

2.5.1 鸿蒙组件使用

UI组件

- `<div>`: 容器组件，用于页面布局
- `<list>` & `<list-item>`: 列表组件，展示天气信息列表
- `<text>`: 文本组件，显示文字信息
- `<image>`: 图片组件，显示天气图标
- `<input>`: 输入组件，城市搜索功能
- `<button>`: 按钮组件，交互操作

自定义组件

- 使用 `<element>` 标签引入自定义组件
- 组件化开发，提高代码复用性
- 12个自定义天气展示组件

2.5.2 鸿蒙API使用

系统API

- `@system.router`: 页面路由导航

- `@system.prompt`: 消息提示（Toast）
- `@system.storage`: 本地数据存储
- `@system.app`: 应用控制（退出应用）
- `@ohos.router`: 路由跳转（新版API）
- `@ohos.hilog`: 日志输出
- `@ohos.app.form.formBindingData`: 卡片数据绑定
- `@ohos.app.form.formInfo`: 卡片信息管理

2.5.3 开发技术栈

前端框架

- 鸿蒙JS UI框架（基于类Vue语法）
- 声明式UI开发
- 数据双向绑定

编程语言

- JavaScript ES6+
- HML（鸿蒙标记语言）
- CSS（样式）
- JSON（配置和数据）

构建工具

- Hvigor（鸿蒙构建系统）
- TypeScript（配置文件）

设计模式

- 单例模式：数据管理类（CityWeatherManager、CityStorageManager）
- 组件化模式：UI组件拆分和复用
- 观察者模式：数据变化驱动UI更新
- 工厂模式：模拟数据生成

2.5.4 数据存储方案

内存缓存

- 天气数据对象缓存
- 缓存时效：30分钟
- 优势：快速访问，减少IO操作

本地持久化

- 使用 `@system.storage` API
- 键值对存储方式
- 存储内容：
  - `currentCity`: 当前城市
  - `savedCities`: 已保存城市列表（JSON字符串）
  - `weather_${cityName}`: 各城市天气数据（JSON字符串）

2.5.5 界面设计规范

布局方案

- Flex弹性布局
- 响应式设计
- 设计宽度：720px

视觉效果

- 毛玻璃效果：`backdropFilter: blur(4px)`
- 圆角设计：20px圆角
- 半透明背景：`rgba(225,225,225,0.5)`
- 渐变和阴影效果

交互设计

- 列表滚动：`scrollpage="true"`
- 点击效果：`clickeffect="false"` 禁用默认效果
- 缓存列表项：`cachedcount="10"` 提升性能

2.5.6 国际化支持

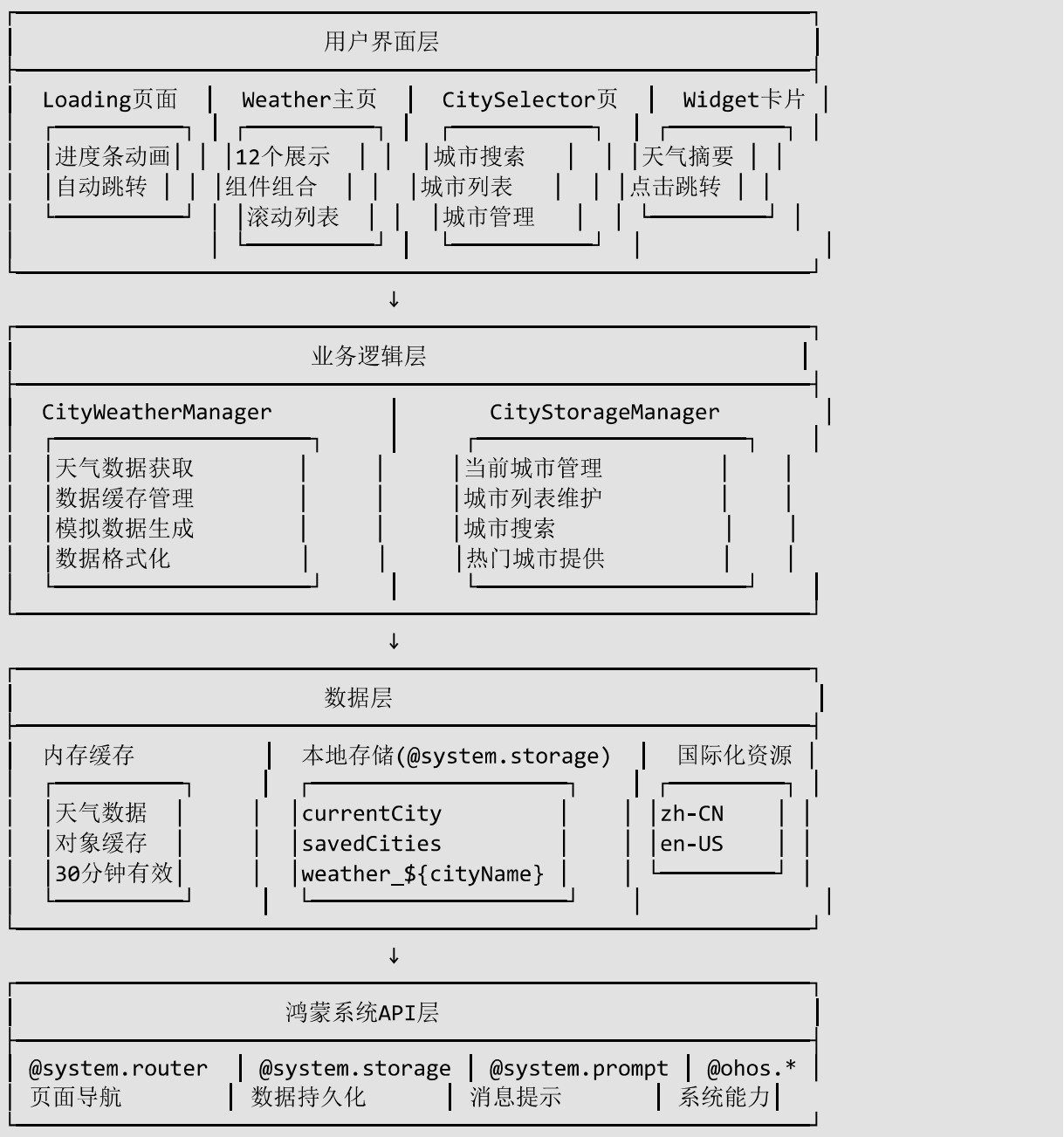
多语言配置

- 中文：`i18n/zh-CN.json`
- 英文：`i18n/en-US.json`
- 资源配置：`resources/base/element/string.json`

支持的语言资源

- 城市选择相关文本
- 界面提示文本
- 应用描述和标签

三、技术架构图



四、功能实现要点

4.1 核心功能实现

天气数据管理

- 采用三级缓存机制：内存缓存 → 本地存储 → 模拟数据生成
- 缓存失效时间：30分钟
- 数据格式统一，便于扩展真实API

城市管理

- 支持最多10个已保存城市
- 新选择的城市自动移至列表首位
- 热门城市预置30个主要城市

数据可视化

- 24小时温度曲线图
- 7天天气趋势展示
- 空气质量指标展示

4.2 用户体验优化

加载体验

- 启动页进度条平滑动画
- 数据加载Toast提示
- 城市切换即时反馈

交互优化

- 双击返回退出应用
- 3秒内再次点击生效
- 列表滚动性能优化（cachedcount）

视觉设计

- 毛玻璃效果增强层次感
- 圆角和半透明设计更柔和
- 统一的视觉语言

4.3 性能优化

渲染优化

- 组件复用，减少重复渲染
- 列表虚拟化（cachedcount设置）
- 数据懒加载

存储优化

- 天气数据按城市分别缓存
- 避免重复请求
- 定期清理过期缓存

内存优化

- 单例模式减少对象创建

- 及时释放不用的数据
- 内存缓存容量控制

五、未来扩展方向

5.1 功能扩展

1. **真实天气API接入**
    - 接入第三方天气数据服务
    - 实现实时数据更新
    - 提供更准确的预报信息
2. **更多生活指数**
    - 花粉过敏指数
    - 运动指数
    - 钓鱼指数
    - 旅游指数
3. **天气预警**
    - 恶劣天气预警推送
    - 自然灾害提醒
    - 温差提醒
4. **个性化设置**
    - 温度单位切换（℃/℉）
    - 主题颜色自定义
    - 字体大小调整
5. **社交分享**
    - 天气卡片分享
    - 生成天气海报
    - 分享到社交平台

5.2 技术优化

6. **离线能力增强**
    - 更长的缓存有效期
    - 离线模式优化
    - 数据同步机制

7. **性能提升**
    - 图片资源优化
    - 代码分包加载
    - 启动速度优化
8. **智能化**
    - AI天气预测
    - 个性化推荐
    - 智能提醒
9. **多端协同**
    - 手机、平板、手表数据同步
    - 跨设备无缝切换
    - 云端数据备份

六、总结

小鸿天气应用是一款基于鸿蒙操作系统开发的天气查询工具，具有以下特点：

技术特色

- 采用鸿蒙JS UI框架，原生体验
- 组件化设计，代码复用率高
- 单例模式管理数据，内存效率高
- 三级缓存机制，响应速度快

功能完善

- 多维度天气信息展示
- 多城市管理和切换
- 生活指数建议
- 桌面卡片服务

用户体验优良

- 界面简洁美观
- 操作流程简单
- 交互反馈及时
- 性能表现优秀

可扩展性强

- 模块化架构易于维护
- 预留API接口易于升级
- 数据格式统一便于扩展

该应用为鸿蒙生态提供了一个优质的天气查询解决方案，展示了鸿蒙开发的技术优势和应用潜力。